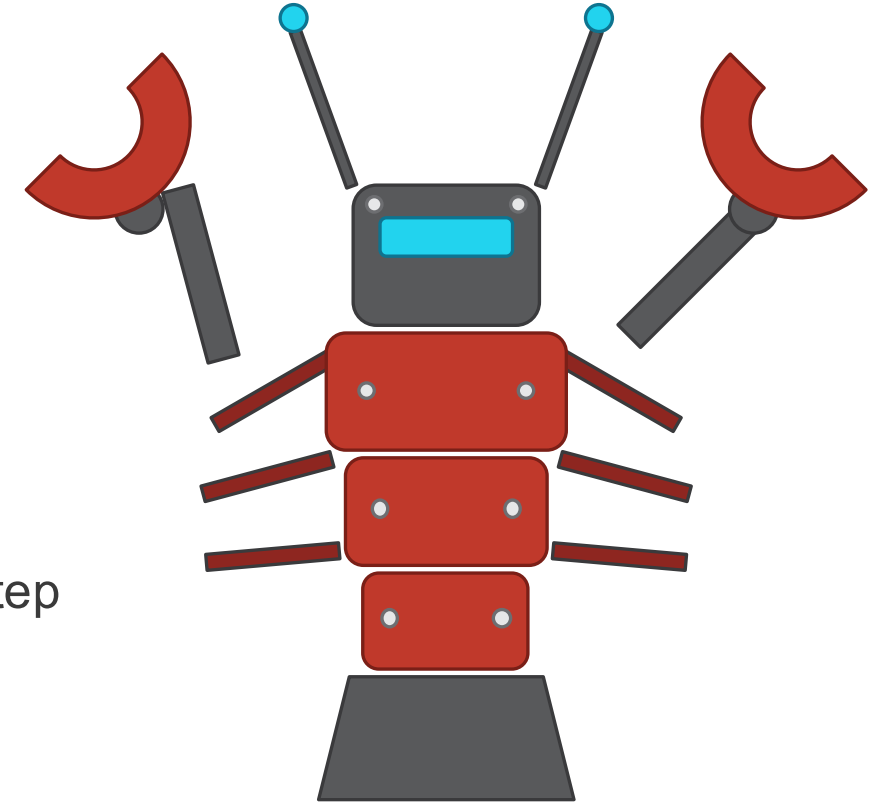
Our Goal

Training a mobile agent that can complete long-horizon multi-step tasks

To carry these out reliably, the agent must be:

- **Grounded** — acts on the right UI element at every step
- **Verifiable** — confirms each state transition matches the intended action



Where GUI agent research is heading

The next stage is verifiable process modeling, not bigger VLMs or more action traces.

Active verification

VAGEN · 2026.01
VeriGUI · 2026.04

Process rewards

Agentic-Q · 2026.02
StainFlow · 2026.06

Memory & state

UI-Mem · 2026.02
SecAgent · 2026.03
MementoGUI · 2026.05
STAMP · 2026.05

World model & recovery

Mobile World Model · 2026.05
RoTS · 2026.05
Teach-and-Repeat · 2026.06

Many independent 2026 works point the same way — verifiable process is the field's consensus next step.

The Core Gap

The field is converging on verifiable process modeling — but today's agents aren't there yet:

Existing GUI agents already possess **strong GUI grounding and single-step/short-steps action prediction**, but they still **lack verifiable state-transition modeling** in long-horizon, multi-branch, noisy environments, which leads to loops, goal drift, failed error recovery, and sparse online RL signals.

→ We break this gap into four concrete challenges.

Four Unsolved Challenges

C1

CHALLENGE 1

Reactive prediction ignores task state

Models optimize whether a click matches the expert, not whether the state actually changed. The result is silent failures.

C2

CHALLENGE 2

Rewards are too sparse or too shallow

Outcome reward is rare on long tasks, and rule rewards stay at the action level. We need reliable step-level credit.

C3

CHALLENGE 3

Memory needs structure, not just text

Screenshot history explodes tokens; text summaries lose visual evidence. Memory must compress yet keep what matters.

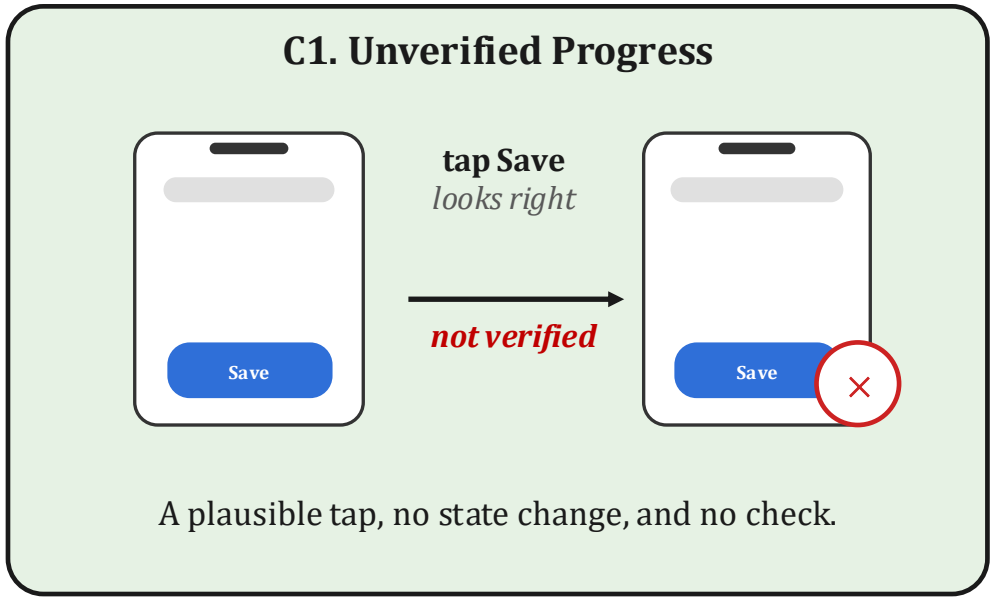
C4

CHALLENGE 4

Recovery is not a training objective

Pop-ups, logins, latency, and mis-taps are common, but data skews to successful runs. Recovery is rarely supervised.

Challenge 1 – Unverified Progress



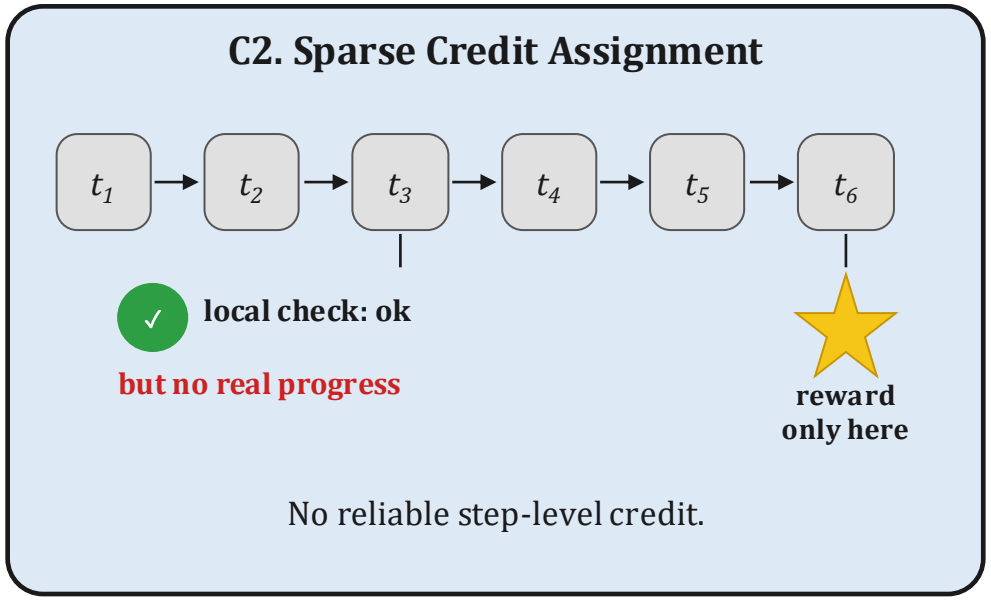
CHALLENGE 1 / 4

Correct clicks, unchecked outcomes

- Action SFT and grounding-RFT supervise the **action itself**: match the expert, hit the right widget. **Whether it took effect is not supervised.**
- The model states neither the **task state** nor the **expected change**, so there is nothing to check the next screen against.
- So failures are **silent**: the tap looks right, the page does not change, and every later step runs on a wrong state.

Needed → declare the task state and expected change, then check them after each action.

Challenge 2 – Sparse Credit Assignment



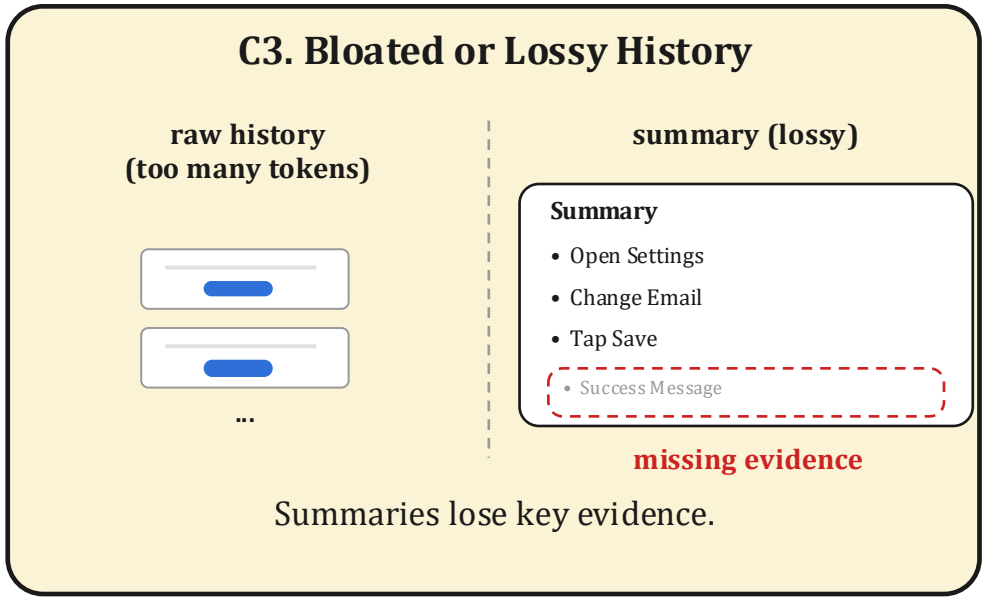
CHALLENGE 2 / 4

Rewards are too sparse or too shallow

- AndroidWorld / MobileGym give **task-level reward** — one success bit at the end of a long task.
- Rule rewards (GUI-R1, Mobile-R1, MobileRL) stabilize training but stay at the **action level**: they reward clicks that look correct.
- VAGEN, Agentic-Q, StainFlow (2026) all point one way: the field needs **step-level credit**.

Needed → dense, reliable step-level credit: reward each transition that truly advances the task.

Challenge 3 – Bloated or Lossy History



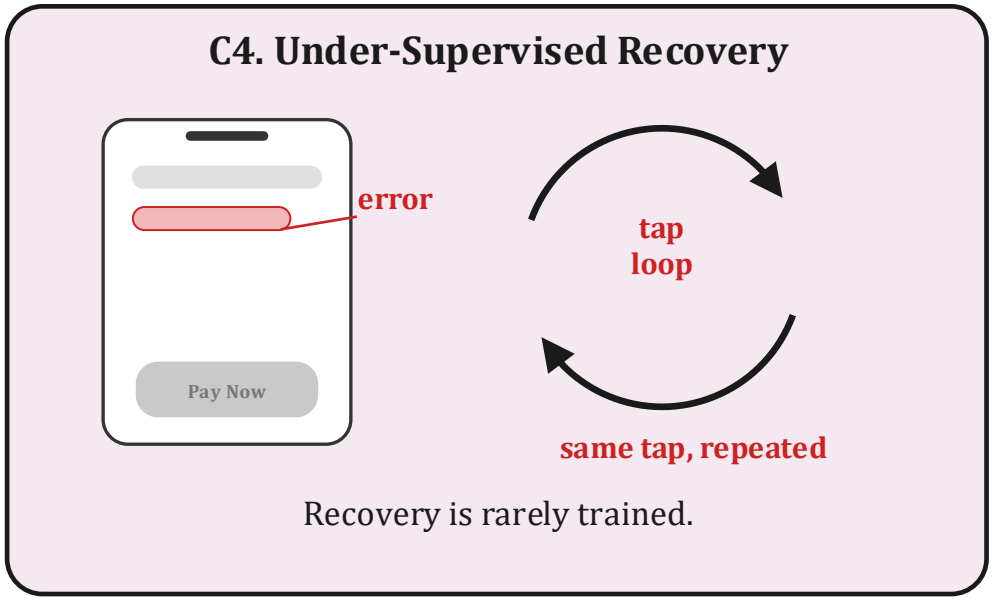
CHALLENGE 3 / 4

Memory needs structure, not just text

- Keeping raw screenshot history blows up the **visual token budget**.
- Pure-text summaries drop **local visual evidence** and transient state (SecAgent, HiconAgent, MementoGUI, STAMP).
- Without structure, the agent cannot answer simple checks: did the popup close? was the field filled?

Needed → a structured, compact state that compresses history but keeps checkable evidence.

Challenge 4 – Under-Supervised Recovery



CHALLENGE 4 / 4

Recovery is not a training objective

- Real phones are noisy: **popups, permissions, ads, logins, keyboard occlusion, network latency, mis-taps**.
- Training data skews to successful expert trajectories — few negatives, almost no recovery demos.
- So agents repeat the same tap instead of clearing the blocker — VeriGUI and RoTS flag recovery as a frontier problem.

Needed → failure and recovery built into data and reward — not left to prompting.

How to Mitigate These Challenges

From predicting the next action to predicting and verifying state transitions.

1 Insight 1 · Make every step checkable

GUI states are observable: OCR, UI tree, page ids, widget states all leave evidence. So have the agent declare state, action, and expected change per step, a contract we can verify. (mitigates C1)

2 Insight 2 · Turn checks into dense rewards

A checked expectation is a step-level signal. Adjacent frames in offline trajectories already imply the labels, so verifiers can score every transition instead of waiting for final success. (mitigates C2)

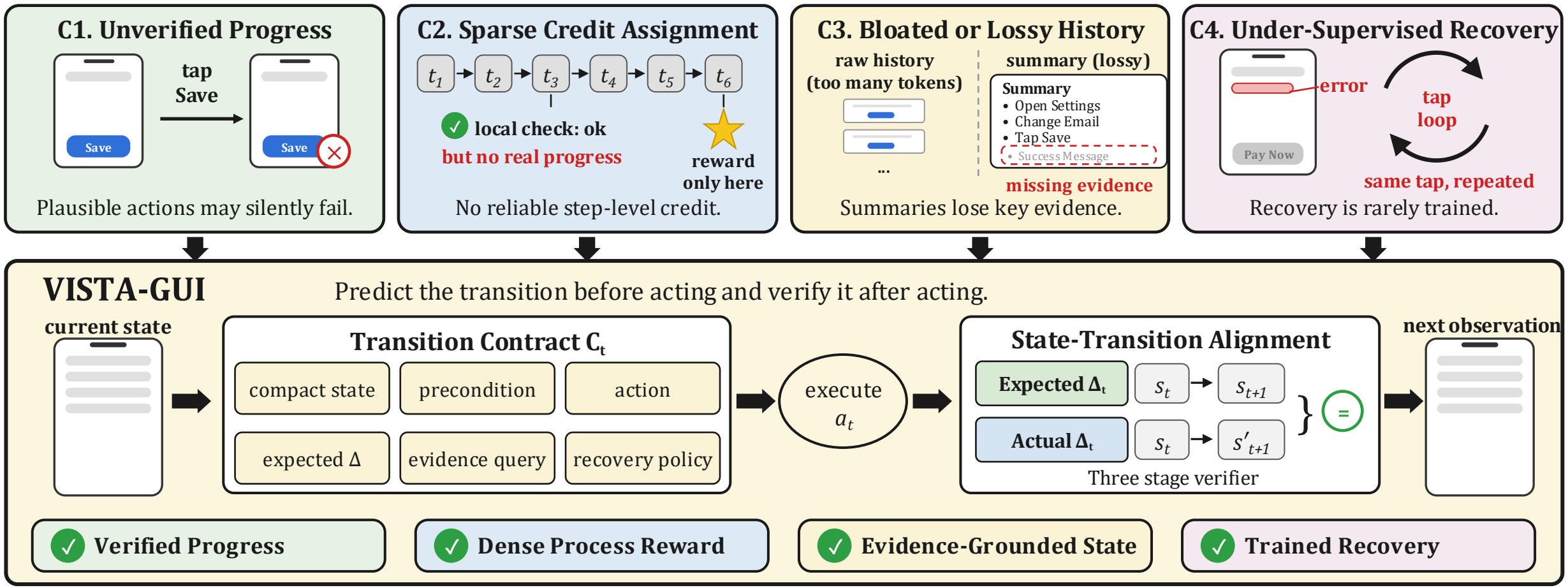
3 Insight 3 · The contract is the state

Screenshot history is too heavy, high-level plans too abstract. The contract keeps task state and evidence compact, compressing history without losing what later steps need. (mitigates C3)

4 Insight 4 · Learn recovery, not just success

Success-only data never teaches that a step failed. We synthesize counterfactual failures and recovery examples, and reward detecting and correcting a failed step. (mitigates C4)

VISTA-GUI (Our Solution)



No such dataset exists, so we build it

Training labels: needed vs. available

	need	public data
state, delta, evidence	✓	✗
failures and no-change steps	✓	✗
recovery demonstrations	✓	✗

no public mobile GUI dataset carries these labels

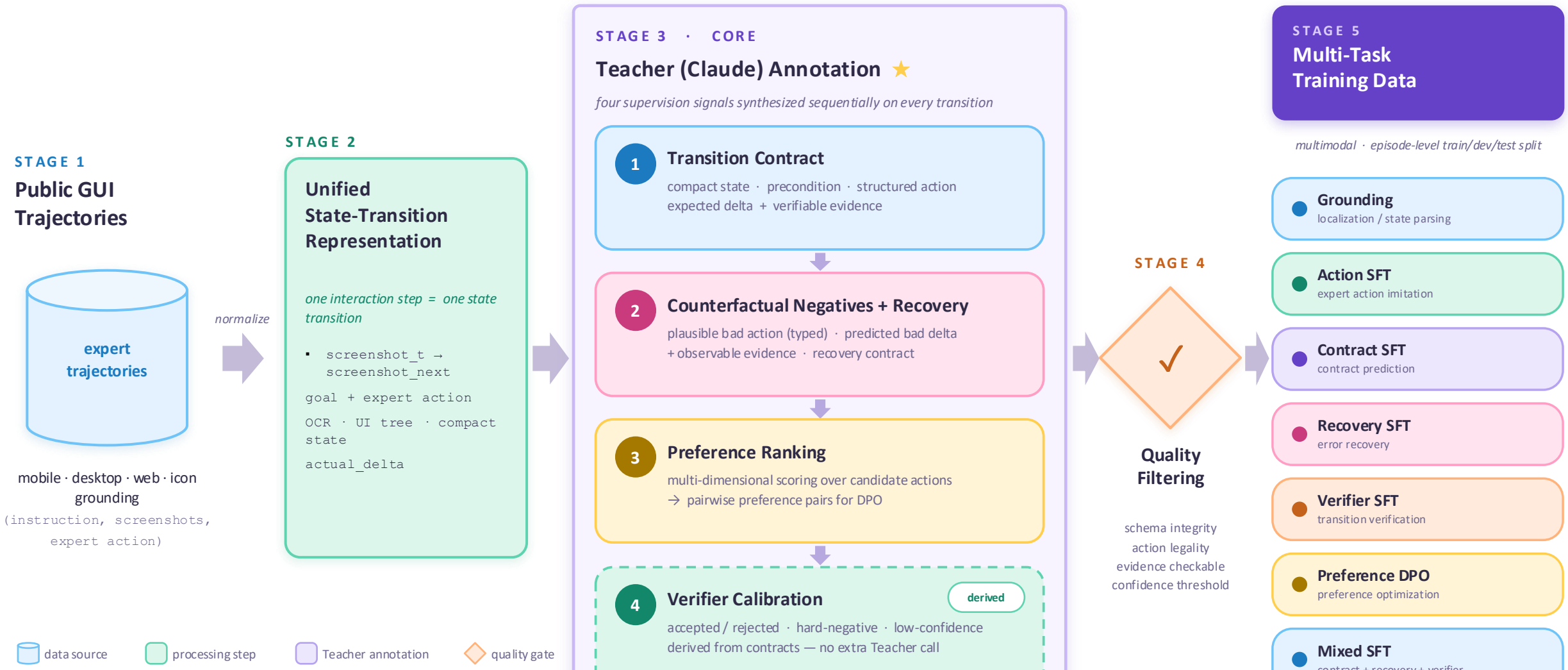
TRAINING DATA

The labels we need do not exist

- Contract training needs **per-step labels**: state, expected change, evidence, plus failures and recovery.
- Public datasets provide **screenshots and expert actions only**, and every trajectory succeeds.
- So we **synthesize the labels**: weak states and deltas from OCR and the UI tree, counterfactual failures, recovery steps.

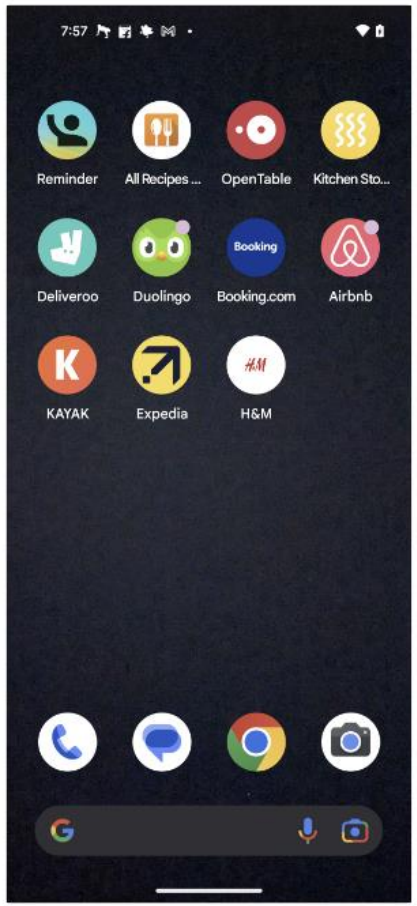
Plan → generate every label automatically from public data.

VISTA-GUI · Data Generation Pipeline



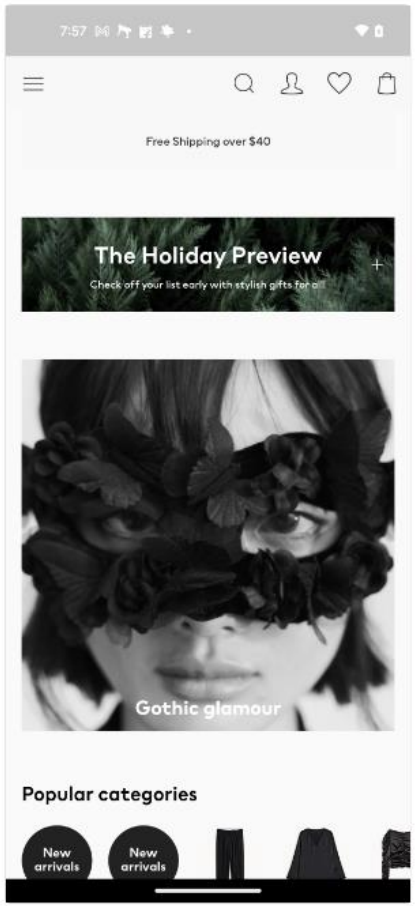
Goal AndroidControl episode 19664 / step 0 open_app

"there is an Important meeting with client and i am left with no fresh formal shirts so ,Search for formal shirts in the H&M app"



(a) State s_t + action

open_app



(b) Next state s_{t+1}

(c) Generated contract annotation

MODEL STATE

page_type: launcher home app: launcher (Android home screen)
summary: Android home/app-drawer screen showing app icons (Reminder, All Recipes, OpenTable, Kitchen Stories, Deliveroo, Duolingo, Booking.com, Airbnb, KAYAK, Expedia, H&M), a bottom dock (Phone, Messages, Chrome, Camera) and a Google search ...
key_elements: H&M app icon visible in the third row; other app icons and dock; Google search bar at bottom

PRECONDITION ✓ satisfied

The device is on the home launcher and the H&M app icon is present, so an open_app('H&M') system action can launch the installed app.
checks: home launcher visible; H&M app icon present on launcher; no blocking dialog obscuring the screen

SEMANTIC ACTION

type: **open_app** app: H&M
target: H&M app
intent: Executes step_instruction 'Open the H&M app' as a preparatory navigation action to reach H&M before searching for ...

EXPECTED DELTA page_changed = true

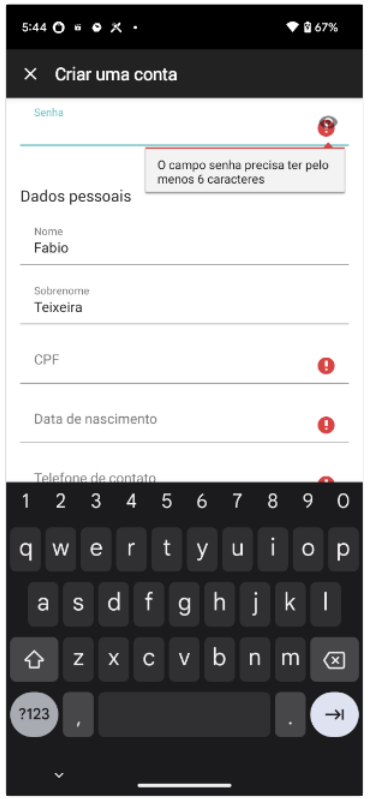
actual_delta: page_changed=true, image_similarity=0.523438; H&M home content appeared and launcher icons disappeared. screenshot_next confirms the H&M home page (search ...
evidence: text_appeared: Free Shipping over $40; text_appeared: The Holiday Preview

RECOVERY BRANCHES

- After open_app the home launcher is still shown (no H&M ... ⇒ tap (0.617, 0.415)
- H&M opened but a startup interstitial/consent/cookie or ... ⇒ tap 'the dialog's dismiss/accept/ ...'
- A blank or loading screen persists after launch. ⇒ wait

Goal AndroidControl episode 8772 / step 0 open_app

"I want to be fit and keep my body strong and improve my health. Set up the account for me as a beginner in the Home workout app."



(a) State s_t + action

open_app →

(b) Next state s_{t+1}

(c) Generated contract annotation

MODEL STATE

page_type: form_page

summary: Current foreground screen is an unrelated app: the Zattini registration form 'Criar uma conta' (Create an account) in Portuguese. A focused password field 'Senha' shows an inline validation error 'O campo senha precisa ter pelo menos ...

key_elements: title 'Criar uma conta'; focused 'Senha' field with validation error tooltip; Nome 'Fabio'

PRECONDITION ✓ satisfied

open_app is a system-level launch action that does not require any particular current screen. It is meaningful here because the target Home Workout app is not in the ...

checks: history_actions is empty (step 0, nothing done yet); screenshot_t foreground is Zattini, not Home Workout; expert_action app_name 'Home Workout' matches the goal app

SEMANTIC ACTION

type: **open_app** app: Home Workout

target: Home Workout app

intent: Launch the Home Workout app to begin the account-setup flow (first step of the task).

EXPECTED DELTA page_changed = true

The Home Workout app comes to the foreground and shows its onboarding 'What's your gender?' screen (Male/Female choice with a NEXT button and a Skip link). The Zattini registration form and ...

evidence: text_appeared: What's your gender?; text_appeared: Let us know you better

RECOVERY BRANCHES

- After open_app the Zattini form (or previous screen) is still ... ⇒ open_app
- Home Workout opens but shows a splash/loading screen rather ... ⇒ wait
- Home Workout opens to an unexpected screen (e.g., a ... ⇒ tap 'dialog dismiss/allow or ...'

VERIFIER HINTS progress = advances task: ... loop risk = low

evidence: +OCR_next contains ''What's your gender?'' +OCR_next contains 'Male' and 'Female' +OCR_next contains 'NEXT' −screen still shows 'Criar uma conta' / ...

notes: open_app of the target app is a single preparatory step; not repetition-prone here.

GUI-Transit Dataset

BUILT ENTIRELY FROM PUBLIC DATA

AndroidControl | AITW | AMEX | CMGUI | MobileViews | ScreenSpot | ScreenSpot-Pro

EXPERT TRAJECTORIES — SCREENS + UI TREES — GROUNDING

automatic weak labels, counterfactuals, recovery

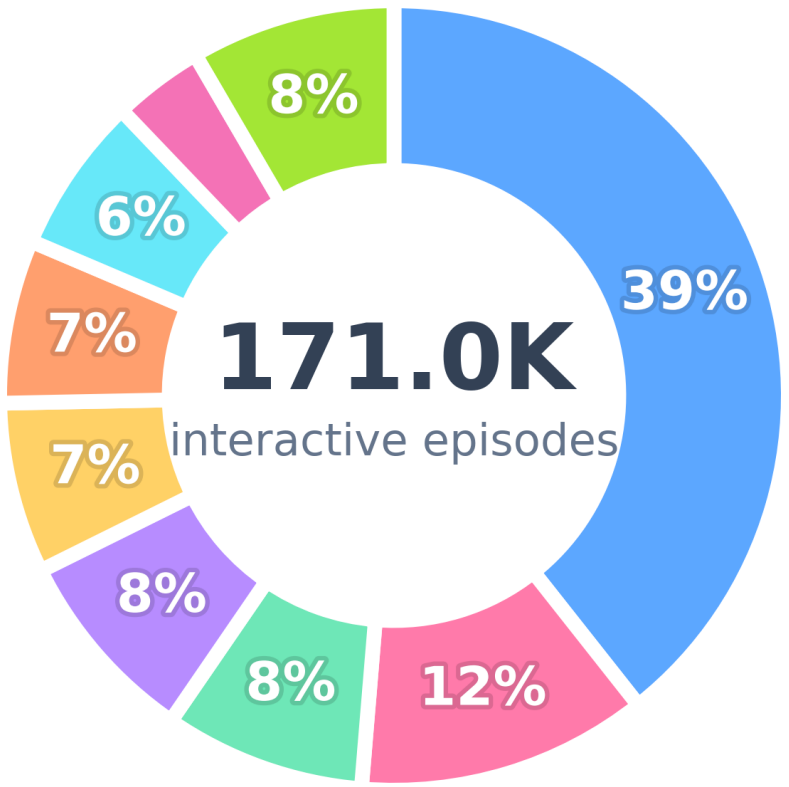
345.9K
transition contracts

119.9K
recovery annotations

59K · 721K
preference states · pairwise judgments

20K
verifier-calibration examples

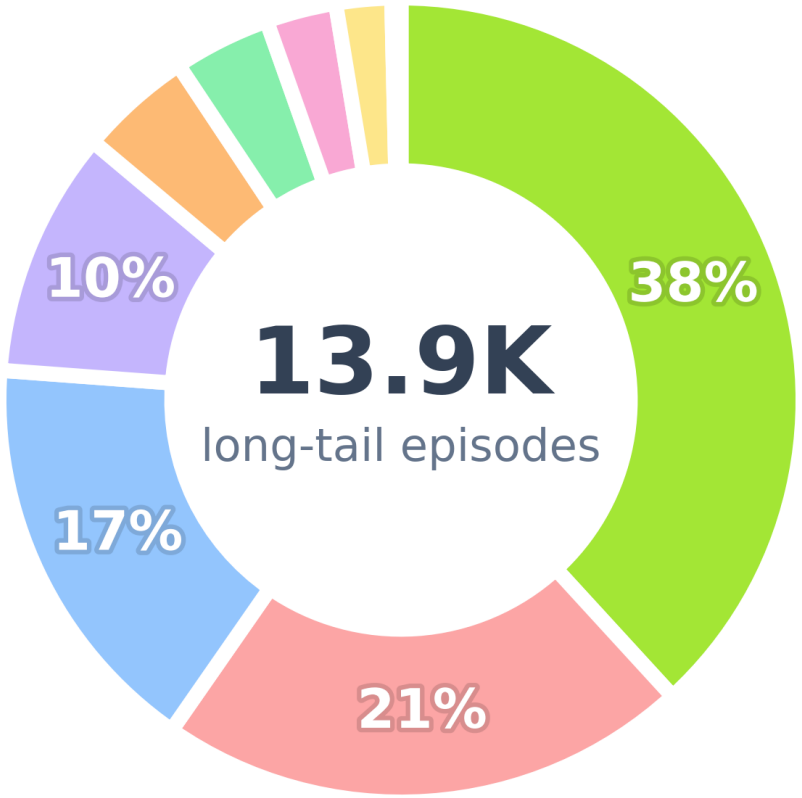
Interactive primary objectives



Top categories

Settings & configuration	67.3K · 39.4%
Shopping & transactions	20.5K · 12.0%
Travel, maps & local	14.1K · 8.3%
Communication & social	13.9K · 8.1%
Media, news & entertainment	12.0K · 7.0%
App launch & management	11.5K · 6.7%
Productivity & scheduling	11.1K · 6.5%
Navigation & browsing	6.4K · 3.7%
Other objectives	13.9K · 8.1%

Interactive objectives: long tail



Long-tail composition

Search & information retrieval	5.5K
Content creation & engagement	3.1K
Account & profile	2.4K
Files & documents	1.4K
Health & fitness	655
Other	563
Education & learning	406
Finance & payments	321
Form & data entry	53

Staged Training Pipeline

SUPERVISED FINE-TUNING

A
Grounding SFT

widget grounding, page parsing & summary

B
Action SFT

next action from screenshot + goal + history

C
Mixed SFT

contract + recovery + verifier SFT

PREFERENCE OPTIMIZATION

D1
DPO

chosen vs. rejected contract pairs

D2
Contract GRPO

groups of contracts → group rewards

MODELS Qwen3.5-9B & Gemma4-E4B-it

Training Data by Stage

Stage	Data sources	Training examples
A · Grounding SFT	MobileViews · AMEX · ScreenSpot · ScreenSpot-Pro	screenshot → widget grounding, page parsing & summary
B · Action SFT	AndroidControl · AITW · AMEX · CMGUI expert trajectories	screenshot + goal + history → next action (click / type / swipe)
C · Mixed SFT	annotated trajectories on the same sources	transition contracts + counterfactual recovery negatives + verifier verdicts
D1 · DPO	sampled contracts scored by deterministic verifiers	chosen vs. rejected contract pairs
D2 · Contract GRPO	Contract-SFT prompts, no fixed targets	groups of sampled contracts scored in-training → group rewards

Evaluation Benchmarks

Benchmark	What it evaluates	Scale	We report
AndroidWorld	main online benchmark: live interaction in real apps with programmatic success checks	116 tasks · 20 real apps	success rate, steps, loop rate, recovery success
ScreenSpot / -Pro	GUI grounding as an isolated sub-capability; Pro adds high-resolution professional UIs	mobile · desktop · web	click accuracy, point-in-box, small-target accuracy
AndroidControl	offline actions and contracts on official splits; high-level vs. low-level instructions	15K demos · 833 apps	action accuracy, step success, state / delta accuracy
GUI-Odyssey	cross-app long-horizon navigation as external validation	7.7K episodes · 201 apps	episode success, cross-app generalization

Contributions

1. **GUI-Transit, the first contract-labeled dataset for mobile GUI agents.** We build approximately 346K transition contracts, 120K recovery annotations, 59K preference-annotated states with 721K pairwise judgments, and 20K verifier-calibration examples, fully automatically from public trajectories, retaining only examples that pass deterministic validity and evidence gates.

2. **Verifiable state-transition alignment.** At every step the agent declares its state, precondition, expected change, and observable evidence, and a three-tier verifier (rules, a lightweight learned model, and limited active probing) checks the declaration against the next screen. Silent failures become detected events, yielding a dense, hack-resistant process reward.

3. **A contract-based training algorithm.** We turn this process reward into policy improvement with a five-stage recipe: Grounding SFT, Action SFT, mixed Contract-Recovery-Verifier SFT, DPO, and Contract GRPO.